Low Level Designing

IAC Provisioning For Finance System

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrollment Number
01	**Mahima Patidar**	**EN22CS301569**
02	**Nikhil Rathore**	**EN22CS301650**
03	**Nitisha Kushwah**	**EN22CS301664**
04	**Nidhish Mehta**	**EN22CS301642**
05	**Moin Husain**	**EN22IT301612**

Group Name: D6 G12

Project Number: DO-23

Industry Mentor Name: Vaibhav

University Mentor Name: Akshay Saxena

Academic Year: 2025-2026

1. Introduction

1.1 Scope of the Document

This document defines the high-level architecture and design for automating infrastructure provisioning in a **Finance System**. It focuses on the use of **Infrastructure as Code (IaC)** tools to create a repeatable, secure, and scalable environment for financial applications.

1.2 Intended Audience

• **Project Supervisors:** To evaluate the technical feasibility and design accuracy.
• **DevOps Engineers:** To understand the automation workflow used for provisioning infrastructure.
• **System Architects:** To review the integration of Kubernetes, Docker, and CI/CD pipelines in the finance platform.

1.3 System Overview

The system automates the deployment of Docker environments and Kubernetes clusters using **Terraform and Jenkins**. By treating infrastructure as code, the system eliminates manual configuration errors and ensures that financial microservices such as **payment processing, account management, and transaction services** are deployed consistently across different environments.

System Design

1.1 Application Design

The application is based on a **Microservices Architecture** where finance-related services such as **Billing, Transaction Processing, Account Management, and Reporting Services** are encapsulated within Docker containers.
These containers are orchestrated by **Kubernetes clusters** to ensure high availability and scalability.

1.2 Process Flow

The process begins at the **Developer's Machine** where IaC scripts are written. The code is pushed to a **Git repository**, which triggers a **CI/CD pipeline (Jenkins)** to execute Terraform plans and apply changes to the Docker host or cloud infrastructure.

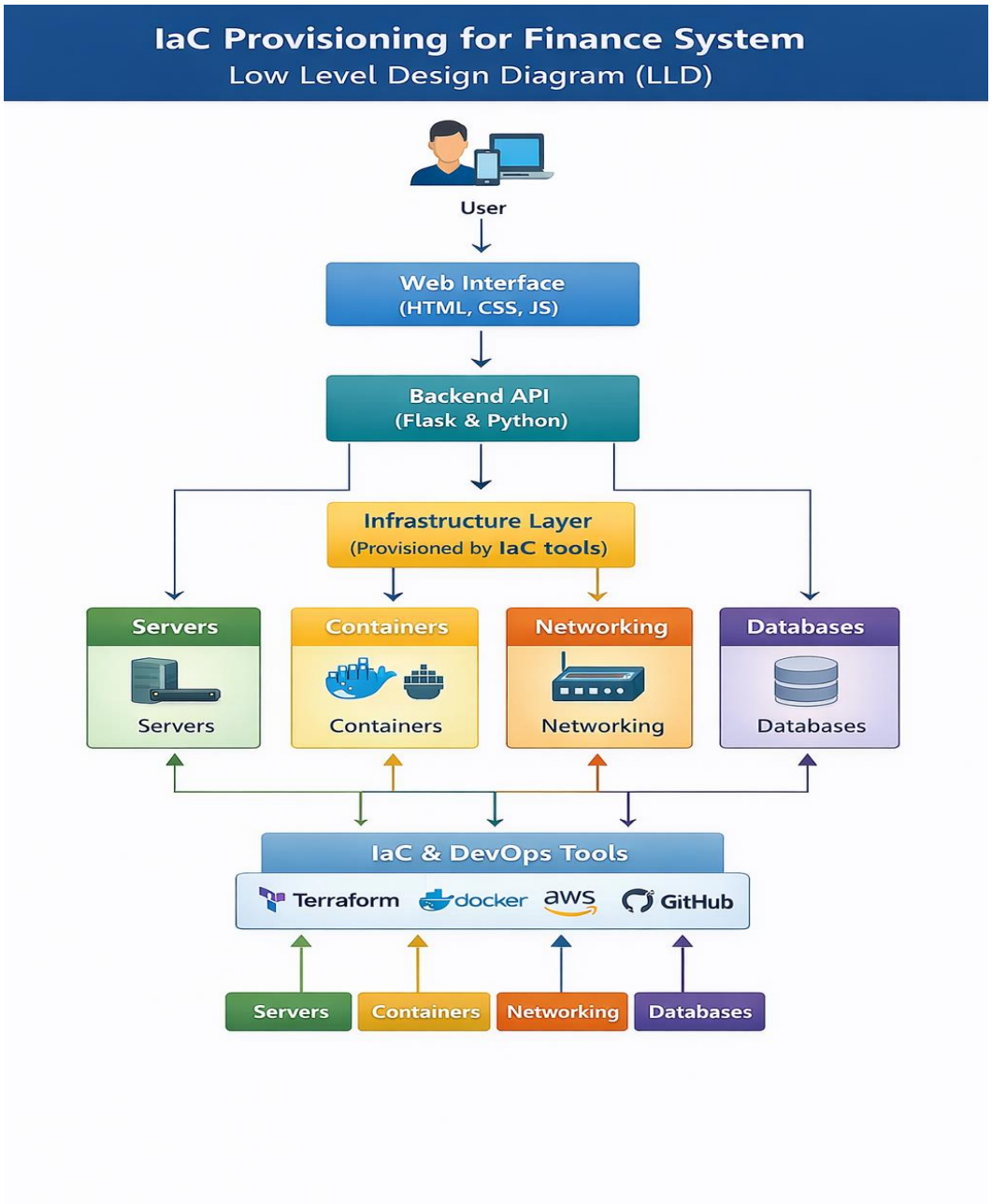

Figure 1: Low-Level System Architecture

1.3 Information Flow

Information flows from the source **(Git repository)** through the automation engine **(Jenkins and Terraform)** to the target infrastructure **(Kubernetes and Docker)**.
Terraform maintains a **State File** to track the mapping between the code and the actual infrastructure resources.

1.4 Components Design

• **Git Repository**
Serves as the central source of truth for infrastructure configuration and application code.

• **Jenkins / CI-CD Pipeline**
Acts as the automation controller that triggers builds and deployments whenever code changes occur.

• **Terraform**
The primary Infrastructure as Code tool used to provision and manage infrastructure resources.

• **Docker & Kubernetes**
Provide the container runtime environment where finance microservices are deployed and managed.

1.5 Key Design Considerations

• **Modularity**
IaC code is divided into modules such as **Network, Compute, Database, and Application Services** for better maintainability.

• **Scalability**
Kubernetes enables the finance platform to scale resources dynamically based on transaction demand.

1.6 API Catalogue

• **Kubernetes API**
Used by the CI/CD pipeline to interact with the cluster and manage the lifecycle of financial service containers.

---

2. Data Design

2.1 Data Model

The project uses an **Infrastructure Meta-Model** to track key entities such as:

• IaC_Modules
• Deployment_Jobs
• Infrastructure_Resources
• Finance_Services

Logical Relationships

1. **Job Triggers Module (1:N)**
   One Jenkins job can execute multiple IaC modules.
2. **Job Creates State (1:1)**
   Every successful deployment job generates exactly one Terraform state version.
3. **Module Defines Resource (1:N)**
   One IaC module can define multiple infrastructure resources.
4. **Resource Hosts Service (1:N)**
   One resource (such as a Kubernetes node) can host multiple finance services.
5. **Resource Belongs to Environment (N:1)**
   Multiple resources are grouped under a specific environment such as **Development, Testing, or Production**.

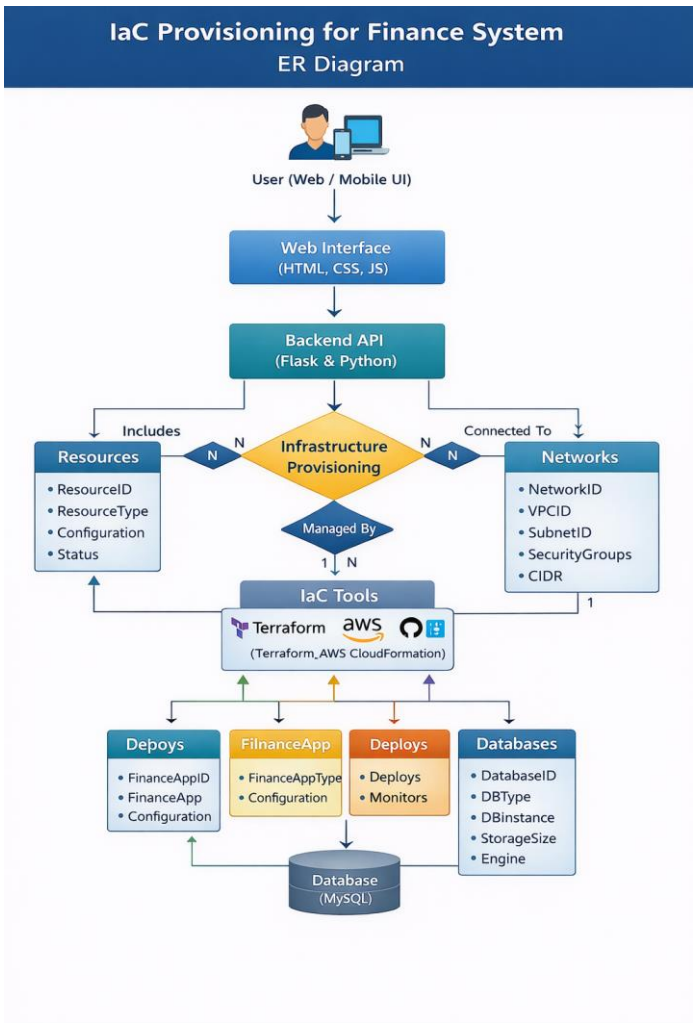

Figure 2: Infrastructure ER Diagram

Data Acess Mechanism

Access to infrastructure code and state files is controlled using **Git repository permissions** and **Terraform backend configurations** such as S3 storage with IAM roles.

2.3 Data Retention Policies

• **State History**
Previous Terraform state versions are retained for backup and disaster recovery.

• **Audit Logs**
Jenkins build logs track who initiated infrastructure changes and when they occurred.

2.4 Data Migration

Infrastructure migration is managed through **Terraform provider configuration updates**, allowing resources to move between **local Docker environments and cloud platforms like AWS**.

---

3. Interfaces

• **User Interface**
Jenkins Web Dashboard for monitoring build and deployment pipelines.

• **Command Line Interface (CLI)**
Terraform CLI and Git CLI used by developers to manage infrastructure scripts and repositories.

---

4. State and Session Management

State management is handled through the **Terraform State File (.tfstate)**.
This file acts as a database that records the current state of the infrastructure and helps Terraform determine which resources must be **created, updated, or destroyed** during deployment.

---

5. Caching

• **Docker Layer Caching**
Jenkins caches Docker layers to speed up container image builds.

• **Terraform Plugin Cache**
Terraform provider plugins are cached locally to reduce deployment time.

6. Functional Requirements

The functional requirements define the services that the **Finance Infrastructure Management Portal** must provide.

6.1 User Authentication & Authorization

• **Registration and Login**
The system must allow authorized administrators to create accounts and log in securely.

• **Password Hashing**
Passwords must be securely stored using **bcrypt hashing**.

• **Session Management**
Secure sessions must be maintained using **JWT (jsonwebtoken)** and **cookie-parser** so that only authenticated users can trigger infrastructure deployments.

6.2 Infrastructure Configuration Management

• **Environment Configuration**
The system should dynamically load environment variables such as **AWS credentials or Docker host details** using dotenv.

• **Script Uploads**
Users should be able to upload infrastructure scripts using **multer**.

• **Database Integration**
Deployment metadata, resource status, and user activities must be stored in **MongoDB** using **mongoose**.

6.3 Provisioning Control & Monitoring

• **Deployment Trigger**
Users should be able to trigger Terraform or Ansible scripts via the Express backend to provision containers or Kubernetes nodes.

• **Real-time Dashboard**
A dynamic interface using **EJS templates** should display the status of financial services and infrastructure.

• **Automated Installation**
The system should automatically install core dependencies on local or remote infrastructure nodes.

6.4 Finance Service Orchestration

• **Microservice Control**
The portal should allow administrators to start, stop, or restart finance microservices such as **Payment Processing, Transaction Service, and Reporting Service**.

• **Resource Mapping**
Infrastructure resources should be mapped to the finance services they host for easy monitoring.

6.5 Dockerfile (Application Containerization)

The **Dockerfile** packages the Finance Management Portal into a portable container image, ensuring that the environment remains identical in development and production stages.

6.6 GitHub Actions (Workflow Configuration)

GitHub Actions automates the **CI/CD pipeline**.
Whenever code is pushed to the main branch, it automatically builds Docker images and runs Terraform validation tests.

6.7 CI/CD Strategy

A. Continuous Integration (CI)

• **Automated Testing**
Each code push triggers automated testing to detect syntax errors in **Node.js and Terraform files**.

• **Security Scanning**
Security tools check for vulnerabilities or exposed secrets in npm packages.

• **Artifact Creation**
After successful testing, a **Docker image** is generated as an immutable deployment artifact.

B. Continuous Deployment (CD)

• **Infrastructure Provisioning**
Terraform updates the infrastructure to match the latest configuration.

• **Container Orchestration**
The Docker image is pushed to a container registry and deployed on Kubernetes clusters.

• **Zero Downtime Deployment**
Updates are rolled out without interrupting financial services such as **payment processing or transaction handling**.

---

7. Non-Functional Requirements

7.1 Security Aspects

• **Automated Installation**
Core dependencies are installed automatically through scripts to avoid configuration vulnerabilities.

• **Secret Management**
Sensitive information such as passwords and API keys are stored securely using **Jenkins credentials or secret managers**.

7.2 Performance Aspects

• **Efficiency**
Automation reduces infrastructure setup time from hours to minutes.

• **Consistency**
IaC eliminates configuration drift and ensures all environments remain identical.

---

8. References

• Problem Statement: **IaC Provisioning for Finance System**
• Project Group: **D6 - Group 12**
• Project Number: **DO-23**

---

9. Github

https://github.com/nidhishmehta03/expense_tracker.git